

2.7 Function Calls

We'll have much more to say about functions in a later chapter, but for now let's just look at how they're called. (To review: what a function *is* is a piece of code, written by you or by someone else, which performs some useful, compartmentalizable task.) You call a function by mentioning its name followed by a pair of parentheses. If the function takes any arguments, you place the arguments between the parentheses, separated by commas. These are all function calls:

```
printf("Hello, world!\n")
printf("%d\n", i)
sqrt(144.)
getchar()
```

The arguments to a function can be arbitrary expressions. Therefore, you don't have to say things like

```
int sum = a + b + c;
printf("sum = %d\n", sum);
```

if you don't want to; you can instead collapse it to

```
printf("sum = %d\n", a + b + c);
```

Many functions return values, and when they do, you can embed calls to these functions within larger expressions:

```
c = sqrt(a * a + b * b)
x = r * cos(theta)
i = f1(f2(j))
```

The first expression squares `a` and `b`, computes the square root of the sum of the squares, and assigns the result to `c`. (In other words, it computes `a * a + b * b`, passes that number to the `sqrt` function, and assigns `sqrt`'s return value to `c`.) The second expression passes the value of the variable `theta` to the `cos` (cosine) function, multiplies the result by `r`, and assigns the result to `x`. The third expression passes the value of the variable `j` to the function `f2`, passes the return value of `f2` immediately to the function `f1`, and finally assigns `f1`'s return value to the variable `i`.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)